

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

Taller Cajero Automático

Modelado UML de un Sistema de Cajero Automático

Programación Avanzada

Grupo: 83

Estudiantes: Juan Sebastian Vasquez Ortiz — 20242020285
Juan Jose Zuñiga Murcia — 20251020043

Profesor: José David Álvarez

Asignatura: Programación Avanzada

21 de septiembre de 2026

Índice

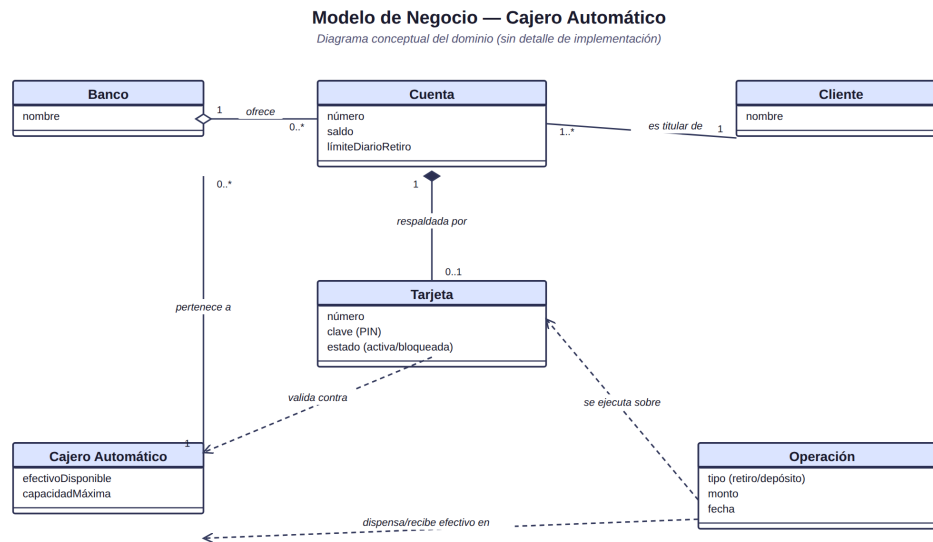
1	Introducción	2
2	Diagramas UML	2
2.1	Modelo de Negocio	2
2.2	Diagrama de Casos de Uso	3
2.3	Diagrama de Clases	4
2.4	Diagrama de Estados	4
2.5	Diagrama de Actividad	5
2.6	Diagrama de Secuencia	6
2.7	Diagrama de Colaboración	7
2.8	Diagrama de Componentes	8
2.9	Diagrama de Despliegue	9
3	Funcionamiento del Programa	9
4	Conclusión	10

1. Introducción

Este informe presenta el modelado UML del sistema *Cajero Automático* desarrollado en Java (Swing), a partir del código fuente ya implementado y funcional. Se incluyen los nueve diagramas solicitados en la guía del taller, contruidos con base en las clases reales del proyecto, seguidos de una explicación del funcionamiento del programa.

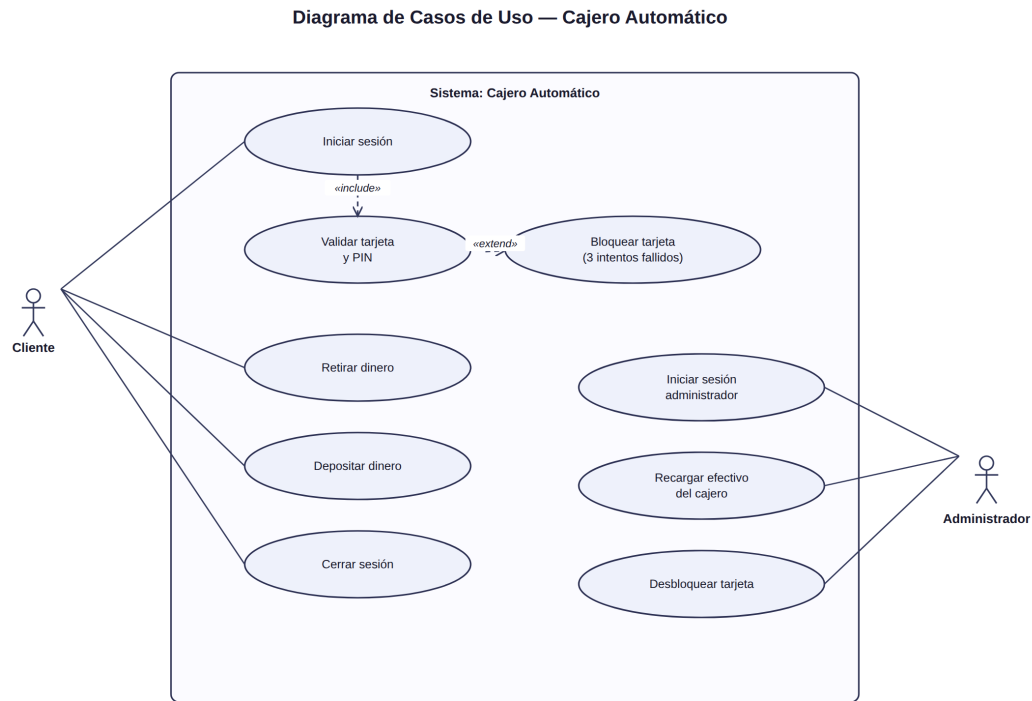
2. Diagramas UML

2.1 Modelo de Negocio



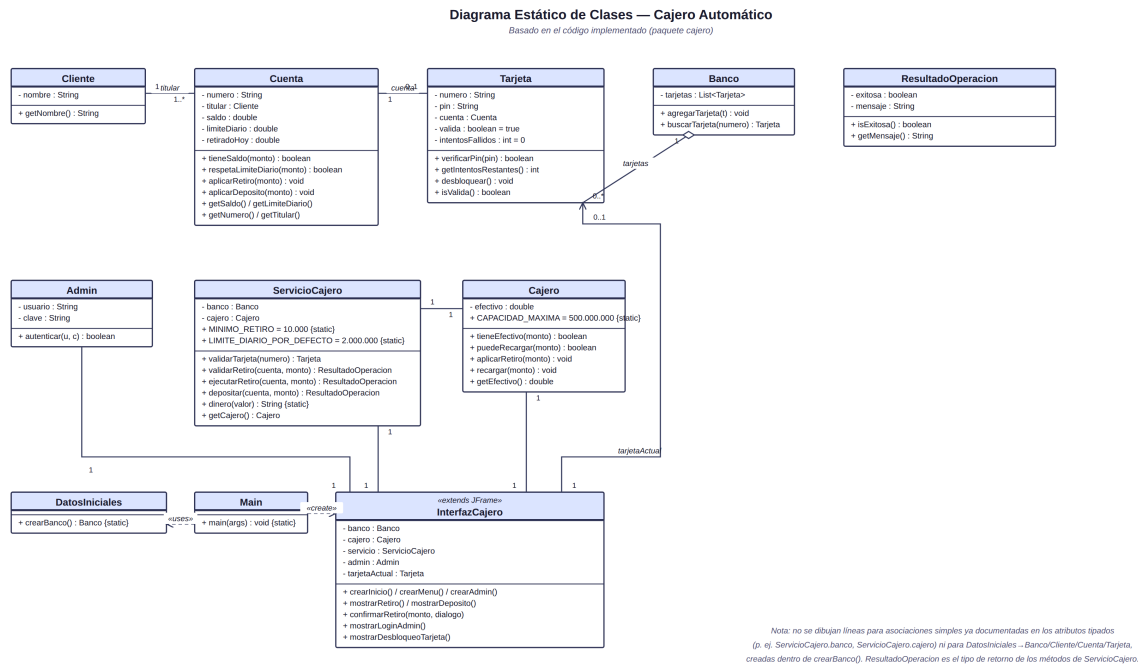
Representa los conceptos del dominio y sus relaciones: un *Cliente* es titular de una o varias *Cuentas*, cada *Cuenta* tiene asociada una *Tarjeta*, y el *Banco* agrupa el conjunto de clientes y cuentas que administra el *Cajero*.

2.2 Diagrama de Casos de Uso



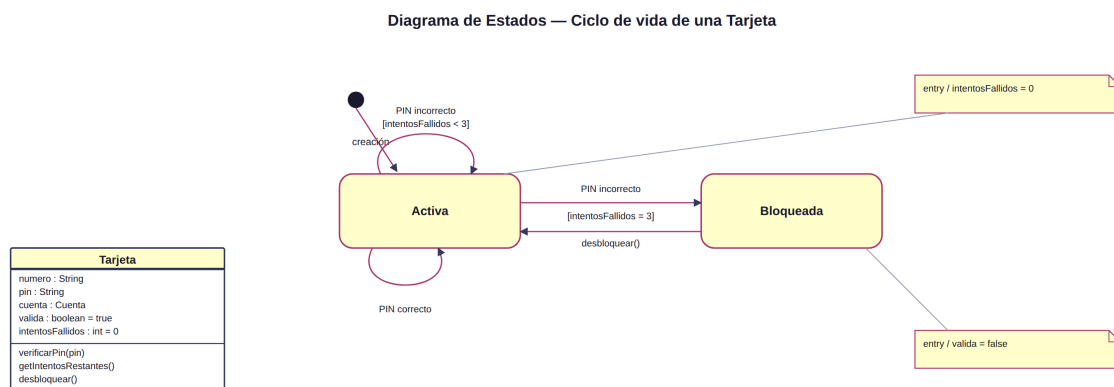
Identifica dos actores: el **Cliente**, que retira y deposita dinero; y el **Administrador**, que recarga efectivo del cajero y desbloquea tarjetas. Los casos de uso del cliente dependen de *Validar tarjeta y PIN*, y este a su vez puede derivar en *Bloquear tarjeta* tras 3 intentos fallidos (relaciones «include»/«extend»).

2.3 Diagrama de Clases



Muestra la estructura estática del sistema: **Main** inicia **InterfazCajero**, que utiliza **ServicioCajero** (la lógica de negocio) para operar sobre **Banco**, **Cajero**, **Cliente**, **Cuenta** y **Tarjeta**. **DatosIniciales** precarga los clientes de prueba y **ResultadoOperacion** encapsula el resultado (éxito/error) de cada operación.

2.4 Diagrama de Estados

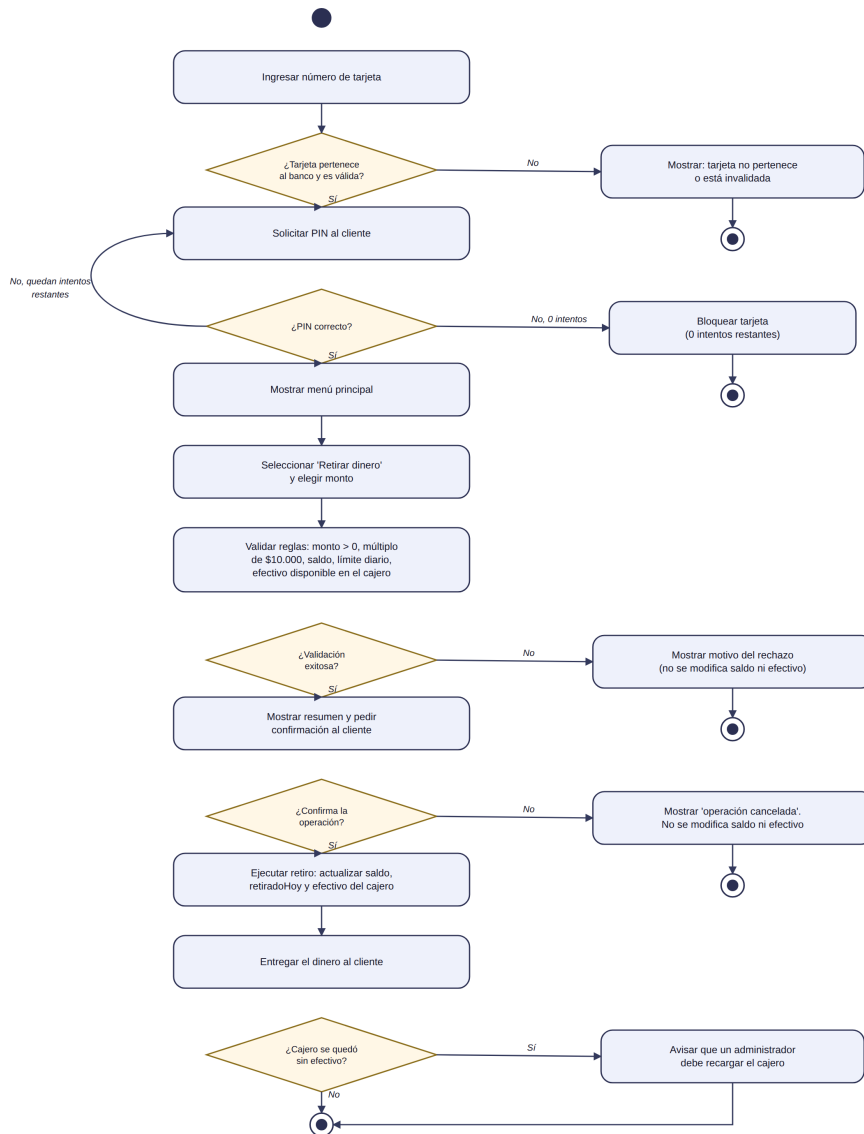


Describe el ciclo de vida de una **Tarjeta**: inicia en estado **Activa**; cada PIN incorrecto incrementa **intentosFallidos**, y al llegar a 3 pasa a **Bloqueada** (**valida** = **false**); el método **desbloquear()**

la regresa a **Activa** reiniciando el contador.

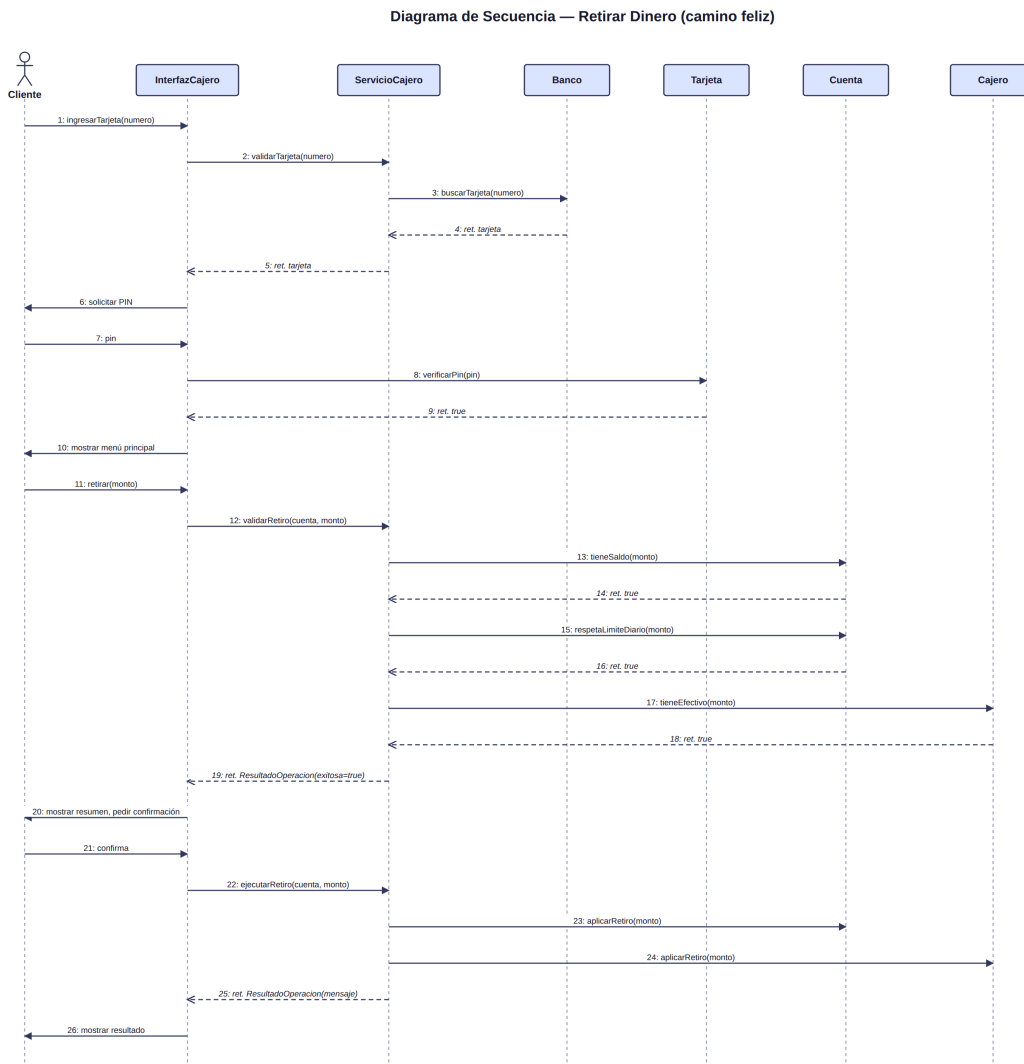
2.5 Diagrama de Actividad

Diagrama de Actividad — Retirar Dinero



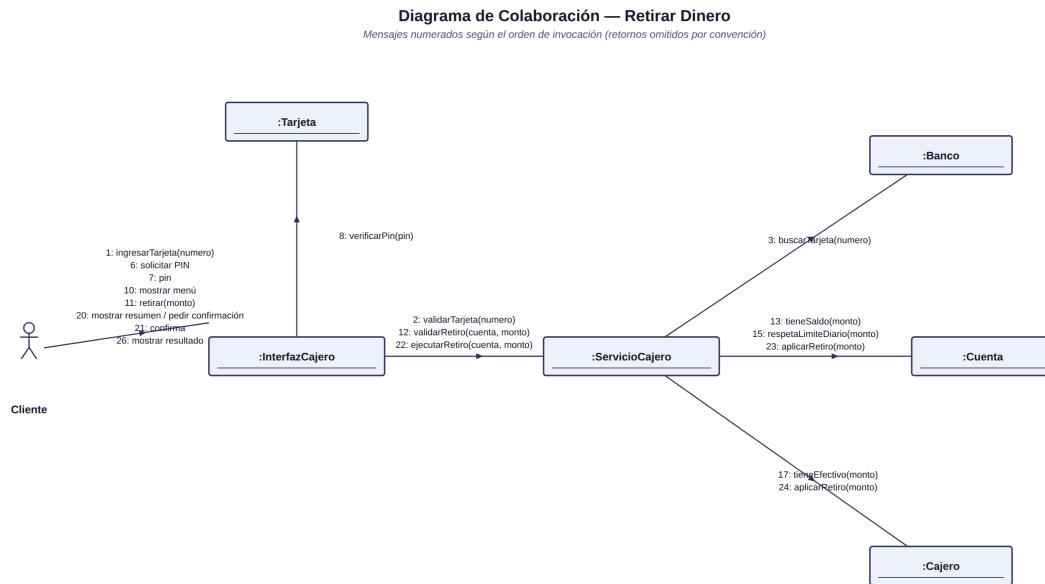
Detalla el flujo de una operación típica (por ejemplo, un retiro): ingreso de tarjeta, validación de PIN (con reintentos y bloqueo), selección de operación, validación de reglas de negocio (saldo, límite diario, efectivo disponible en el cajero) y despliegue del resultado.

2.6 Diagrama de Secuencia



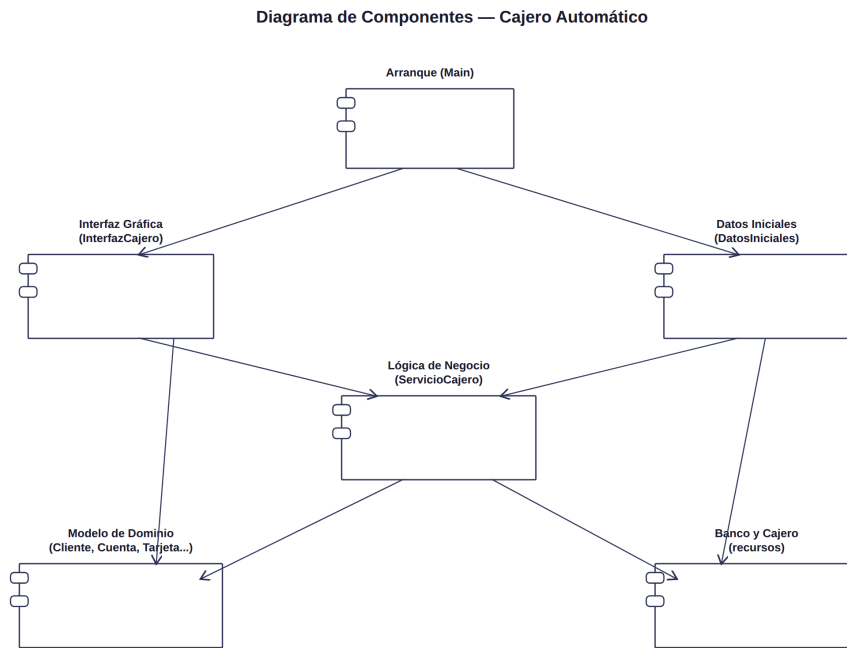
Ilustra la interacción temporal entre **Cliente**, **InterfazCajero**, **ServicioCajero**, **Tarjeta**, **Cuenta**, **Banco** y **Cajero** durante un retiro exitoso: desde el login hasta la actualización del saldo y la entrega del efectivo.

2.7 Diagrama de Colaboración



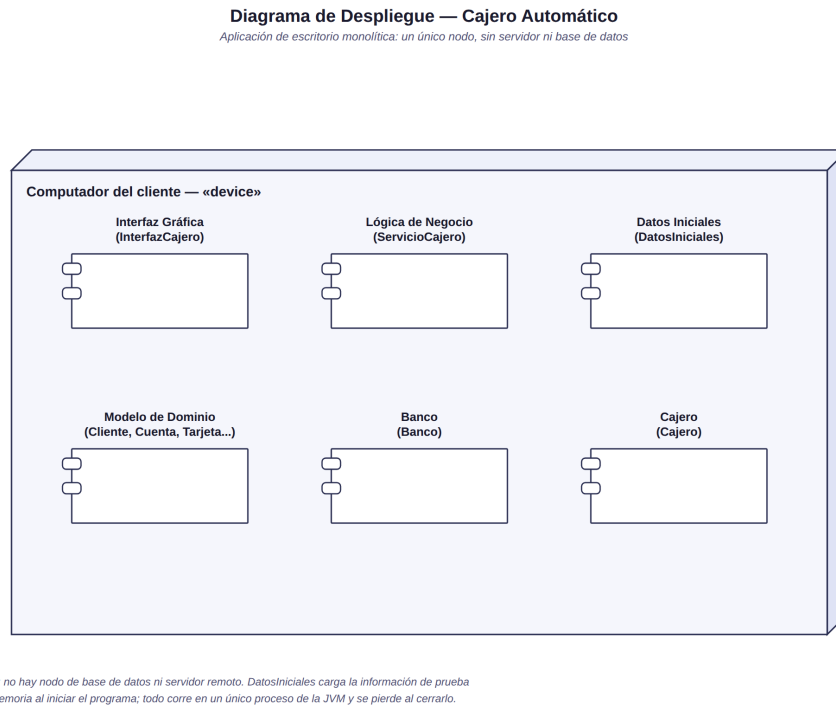
Presenta la misma operación de retiro que el diagrama de secuencia, pero enfatizando los enlaces entre objetos y el orden numerado de los mensajes en lugar de la línea de tiempo.

2.8 Diagrama de Componentes



Agrupar el sistema en seis componentes: *Arranque (Main)*, *Interfaz Gráfica*, *Datos Iniciales*, *Lógica de Negocio*, *Modelo de Dominio* y *Banco y Cajero*, con las dependencias de invocación entre ellos.

2.9 Diagrama de Despliegue



El sistema corre en un único nodo (*Computador del cliente*), sin servidor ni base de datos: es una aplicación de escritorio que carga sus datos de prueba en memoria al iniciar y los pierde al cerrarse.

3. Funcionamiento del Programa

Al iniciar, **Main** carga **DatosIniciales** (tres clientes de prueba con tarjeta, PIN y saldo precargados) y abre la ventana principal (**InterfazCajero**).

Autenticación. El usuario ingresa el número de tarjeta y el PIN. **ServicioCajero** valida ambos contra los datos del banco. Si el PIN es incorrecto, se incrementa el contador de intentos fallidos de la tarjeta; al tercer fallo la tarjeta se bloquea (**valida = false**) y deja de aceptar operaciones hasta ser desbloqueada por un administrador.

Operaciones del cliente. Con sesión activa, el menú ofrece dos operaciones (además de cerrar sesión):

- **Retirar dinero:** se valida que el monto sea múltiplo de \$10.000, que no supere el saldo disponible ni el límite diario, y que el cajero tenga efectivo suficiente. El saldo antes y después del retiro se muestra como confirmación.
- **Depositar dinero:** se valida que el monto sea múltiplo de \$10.000 y positivo; se actualiza el saldo de la cuenta.

Panel de administrador. Tras autenticarse (usuario y contraseña), el administrador puede **recargar el efectivo** del cajero (respetando su capacidad máxima de \$500.000.000) y **desbloquear tarjetas** que quedaron bloqueadas por PIN incorrecto.

Por qué números de tarjeta arbitrarios no funcionan. El sistema no genera ni acepta tarjetas

al azar: **Banco** solo reconoce las tarjetas precargadas por **DatosIniciales** al iniciar el programa. Cualquier número distinto a los de prueba no existe en memoria y por lo tanto **ServicioCajero** responde que la tarjeta no pertenece al banco.

4. Conclusión

El sistema implementa correctamente las reglas de negocio de un cajero automático básico (autenticación con bloqueo por intentos fallidos, límites de retiro y control de efectivo), y los diagramas UML presentados reflejan fielmente la estructura y el comportamiento del código fuente actual.